EMERGEX

Centralised Student Accommodation & Secure Booking Application

Phase 5 — Implementation

IT Project 700 · Semester 2, 2026

Group 55

Due 9 November 2026

Group members

Student no.	Name	Surname	Email	Contact no.
402308936	Tristan	Moses	402308936@my.richfield.ac.za	082 801 5082
402411668	Reenesh	Ramawthar	402411668@my.richfield.ac.za	084 604 1521
402411227	Joyce	Kamalu	402411227@my.richfield.ac.za	067 107 5365
402412304	Novithan	Naidoo	402412304@my.richfield.ac.za	062 082 9186
402413409	Thingo	Shezi	402413409@my.richfield.ac.za	071 429 9251
402111846	Lungelo	Ngwane	402111846@my.richfield.ac.za	064 253 4563

Contents

5.1 Introduction

5.1.1 Purpose of this phase

5.1.2 What was implemented

5.1.3 Technology stack

5.1.4 Implementation approach

5.1.5 Project boundary

5.2 Coding

5.2.1 Architecture

5.2.2 Coding conventions

5.2.3 Representative modules

5.2.4 Security implementation

5.2.5 Known limitations in the delivered code

5.3 Testing

5.3.1 Objectives

5.3.2 Scope

5.3.3 Test strategy — the six levels

5.3.4 Test environment

5.3.5 Tools

5.3.6 Framework architecture

5.3.7 Test design principles

5.3.8 Entry and exit criteria

5.3.9 Executing the suite

5.4 System Testing

5.4.1 Test case design

5.4.2 Case status and execution result

5.4.3 Test run record

5.4.4 The 36 test cases and their outcomes

5.4.5 Execution results

5.4.6 Defects identified

5.4.7 Evaluation of the testing results

5.4.8 Limitations

5.4.9 Recommendations

5.5 Installation

5.5.1 Requirements

5.5.2 Installation procedure

5.5.3 Demonstration accounts

5.5.4 Verifying the installation

5.5.5 Troubleshooting

5.5.6 Resetting and backing up

5.5.7 Deployment considerations

Conclusion

Appendix A — Test case specifications

Appendix B — Reproducing the test run

5.1 Introduction

5.1.1 Purpose of this phase

The Implementation phase turns the design produced in Phase 4 into a working system. Where earlier phases described what EMERGEX should do and how it should be structured, this phase documents what was actually built, how it was proven to work, and how it is installed and run.

The phase has five parts. Section 5.1 sets out what was delivered and how the work was approached. Section 5.2 describes the code. Sections 5.3 and 5.4 cover testing: the strategy, and then the test cases, their execution and the defects found. Section 5.5 is the installation procedure.

5.1.2 What was implemented

EMERGEX is a web application that connects students with verified accommodation, addressing accommodation fraud through three mechanisms: landlord verification by an administrator, listing verification before publication, and a deposit held in escrow until the student has physically checked in.

Table 5.1.1 — Delivered functionality by role

Role	Implemented capability
Public visitor	Browse published listings; search by city, suburb or nearby institution; filter by NSFAS accreditation, maximum rent and instant-book availability; view listing detail with verification badges
Student	Register and sign in; book instantly or submit a request; track bookings through their lifecycle; simulate a deposit hold; confirm check-in; upload NSFAS evidence for review
Landlord	Register; view listings; review incoming booking requests within a 12-hour response window; approve or decline
Administrator	Verify landlords; verify and publish listings; review uploaded documents; inspect an audit trail; generate operational reports with CSV export

Alongside the application, an automated test suite of 118 tests across sixlevels was built to verify it. That suite is a deliverable of this phase in its own right and is described in sections 5.3 and 5.4.

5.1.3 Technology stack

Table 5.1.2 — Implementation technologies

Layer	Technology	Reason
Web server	Apache (XAMPP)	Prescribed for the module; matches the assessment environment
Application	PHP 8.1+, no framework	A framework would hide the mechanisms being assessed — routing, sessions, CSRF and escaping are all visible in the source
Database	MySQL 8 / MariaDB, InnoDB	InnoDB is required for foreign keys and transactions: the booking insert and its escrow insert must both succeed or both fail
Database access	PDO, native prepared statements	`ATTR_EMULATE_PREPARES => false` sends parameters separately from the statement, which is what actually prevents SQL injection
Front end	Hand-written HTML, CSS, JavaScript	No build step and no dependencies to install or break
Test automation	C# · NUnit 4.6 · Playwright 1.62 .NET 9	See section 5.3.5

5.1.4 Implementation approach

Development proceeded feature by feature rather than layer by layer: each capability was taken from database schema through to working page before the next was started. This keeps every increment demonstrable and surfaces integration problems early — a schema that cannot support the page is discovered in the same sitting rather than weeks later.

Security was implemented as the application was written, not retrofitted. Prepared statements, output escaping, CSRF tokens and role checks are present in the first version of each handler. Retrofitting security means auditing every line already written, and anything missed stays missed.

> **A note on sequencing.** The prototype was built ahead of the prescribed phase order, and a working system existed before the analysis and design documentation was finalised. This was deliberate, to reduce delivery risk on the most heavily weighted phases.
>
> One consequence is worth stating plainly: defects found during testing have been fed back into the earlier documentation rather than recorded only at the end. The missing room-availability constraint appears in the data integrity discussion, and the document storage weakness in the security design. The phase documents therefore describe the system as it is, including its known faults.

5.1.5 Project boundary

The escrow feature is a simulator. No payment service provider is integrated and no funds move at any point. A production system would require a regulated payment provider together with legal, security,

privacy and financial compliance review — none of which is in scope for an academic prototype.

For the same reason, real identity or NSFAS documents must never be uploaded to this system. The upload feature exists to demonstrate the review workflow, and section 5.4.6 records a confirmed weakness in how uploaded files are served.

5.2 Coding

5.2.1 Architecture

EMERGEX uses a page-per-URL model: each user-facing address maps to one PHP file in `public/`, and every one of those files begins by including a shared bootstrap. The bootstrap establishes the session, sends security headers, exposes the database connection and provides the helper functions every page relies on.

This is deliberately simple. A router and MVC layer would be more elegant, but would also place three layers of indirection between a request and the code that answers it — and the module assesses understanding of that code.

```
application/
├── app/
│   ├── config/config.php          Database, base URL, upload limits
│   └── includes/
│       ├── bootstrap.php          Session, headers, PDO, helpers
│       ├── header.php             Shared page head and navigation
│       └── footer.php             Shared page foot
├── database/
│   ├── schema.sql                 Tables, constraints, indexes
│   ├── seed.sql                   Demonstration data
│   └── install.php                CLI installer
├── public/
│   ├── index.php                  Landing page
│   ├── properties.php             Search and filtering
│   ├── property.php               Listing detail
│   ├── book.php                   Booking creation
│   ├── dashboard.php              Student workflow
│   ├── landlord.php               Landlord workflow
│   ├── admin.php                  Verification and audit
│   ├── reports.php                Reports and CSV export
│   ├── login.php / register.php / logout.php
│   ├── assets/                    CSS, JavaScript, images
│   └── uploads/                   Uploaded documents
└── tests/
    ├── security_checks.php        Static security checks
    └── acceptance_tests.php       PHP acceptance suite
```

Only `public/` should be exposed by the web server in a production deployment. Configuration, database scripts and includes sit outside it so they cannot be requested directly.

5.2.2 Coding conventions

- **`declare(strict_types=1)` in every file.** PHP otherwise coerces types silently — a string where an integer was expected becomes `0` rather than an error, which hides bugs.
- **Every database query is a prepared statement.** There is no concatenation of user input into SQL anywhere in the codebase.

- **Every variable echoed into HTML passes through `e()`.** Unescaped output is how stored content becomes executable script.
- **Every state-changing POST verifies a CSRF token** before doing anything else.
- **Authorisation is checked on the server for every request**, never inferred from what the interface chose to display.
- **Money is `DECIMAL`, never float.** Binary floating point cannot represent decimal fractions exactly.

5.2.3 Representative modules

Six modules are described below. Between them they cover configuration and security, the main read path, the main write path, the state machine, the administrative workflow, and the datalayer.

Module 1 — `app/includes/bootstrap.php`

Included first by every page. It configures the session, sends security headers, and provides the shared helpers.

```
session_name('emergex_session');
session_set_cookie_params([
    'httponly' => true,                                  // not readable from JavaScript
    'secure'   => !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off',
    'samesite' => 'Lax',                                 // not sent on cross-site POSTs
    'path'     => '/',
]);
session_start();

header('X-Frame-Options: DENY');                         // anti-clickjacking
header('X-Content-Type-Options: nosniff');               // no content-type guessing
header('Referrer-Policy: strict-origin-when-cross-origin');
header("Content-Security-Policy: default-src 'self'; ... frame-ancestors 'none'");
```

The PDO connection is created once and reused through a static variable, with three options that matter:

```
$pdo = new PDO($dsn, $db['user'], $db['password'], [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,  // fail loudly, never silently
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,                   // real prepares
]);
```

`ATTR_EMULATE_PREPARES => false` is the important one. With emulation on, PDO builds the final SQL string itself and only *appears* parameterised. With it off, parameters travel to MySQL separately from the statement, so user input can never be parsed as SQL.

The file also provides `csrf_token()`, `verify_csrf()`, `e()`, `require_auth()`, `audit()`, `money()` and the login throttle.

Module 2 — `public/properties.php` (search)

The main read path. Filters build a parameterised WHERE clause: the *shape* of the query varies with the filters chosen, but every value supplied by the user is bound rather than interpolated.

```
$where  = ["p.status = 'published'", 'p.verified_at IS NOT NULL'];
$params = [];

if ($keyword !== '') {
    $where[]  = '(p.city LIKE ? OR p.suburb LIKE ? OR p.near_institution LIKE ?)';
    $like     = '%' . $keyword . '%';
    $params[] = $like; $params[] = $like; $params[] = $like;
}
if ($maxRent > 0) { $where[] = 'p.monthly_rent <= ?'; $params[] = $maxRent; }
if ($nsfasOnly)   { $where[] = 'p.nsfas_accredited = 1'; }

$stmt = db()->prepare('SELECT ... WHERE ' . implode(' AND ', $where) . ' ORDER BY ...');
$stmt->execute($params);
```

The two conditions always present — `status = 'published'` and `verified_at IS NOT NULL` — enforce at query level that unverified or unpublished listings can never reach a visitor, whatever filters are supplied.

Module 3 — `public/book.php` (booking creation)

The main write path, and the clearest example of a transaction. A booking and its escrow record must both exist or neither must.

```
$pdo = db();
$pdo->beginTransaction();
try {
    $status   = $property['instant_book'] ? 'approved' : 'pending';
    $deadline = $property['instant_book'] ? null
              : (new DateTimeImmutable('+12 hours'))->format('Y-m-d H:i:s');

    $insert = $pdo->prepare('INSERT INTO bookings
        (property_id, student_id, move_in_date, message,
         monthly_rent, deposit_amount, status, response_deadline, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, NOW())');

    // Money comes from the property row, never from the request.
    $insert->execute([$propertyId, $user['id'], $moveIn, $message,
        $property['monthly_rent'], $property['deposit_amount'], $status, $deadline]);

    $bookingId = (int) $pdo->lastInsertId();

    $escrow = $pdo->prepare('INSERT INTO escrow_transactions
        (booking_id, amount, status, created_at) VALUES (?, ?, "not_paid", NOW())');
    $escrow->execute([$bookingId, $property['deposit_amount']]);

    $pdo->commit();
    audit('booking.created', 'booking', $bookingId);
} catch (Throwable $exception) {
    $pdo->rollBack();
    $errors[] = 'The booking could not be created. Please try again.';
}
```

Two decisions are worth drawing out. **Rent and deposit are read from the property row**, not from the submitted form — a tampered price field has no effect, verified by test case TC-30. **The caught exception produces a generic message**; the underlying error is never shown, because database errors disclose table and column names.

Module 4 — `public/dashboard.php` (escrow state machine)

The student dashboard advances the escrow simulation. Every action re-queries the booking *joined to the current user*, so ownership is proven by the query rather than assumed:

```
$stmt = db()->prepare('SELECT b.*, e.id AS escrow_id, e.status AS escrow_status
    FROM bookings b
    JOIN escrow_transactions e ON e.booking_id = b.id
    WHERE b.id = ? AND b.student_id = ?');          // ownership enforced here
$stmt->execute([$bookingId, $user['id']]);
$booking = $stmt->fetch();

if (!$booking) { http_response_code(404); exit('Booking not found.'); }
```

A booking belonging to someone else returns **404, not 403**. That is intentional: 403 would confirm the booking exists, telling an attacker that a given identifier is real. 404 reveals nothing. Test case TC-33 verifies this.

The same file handles NSFAS uploads, validating by content rather than by file name:

```
$mime = (new finfo(FILEINFO_MIME_TYPE))->file($file['tmp_name']);   // inspect the bytes

if (!in_array($mime, config('uploads.allowed_mime'), true)) {
    flash('error', 'Only PDF, JPG and PNG files are accepted.');
} else {
    $extension = match ($mime) {                        // extension from content
        'application/pdf' => 'pdf',
        'image/jpeg'      => 'jpg',
        'image/png'       => 'png',
    };
    $stored = bin2hex(random_bytes(20)) . '.' . $extension;   // unguessable name
    move_uploaded_file($file['tmp_name'], $target);
}
```

The client-supplied file name never builds the storage path, which is why a name containing directory traversal sequences has no effect (TC-34).

Module 5 — `public/admin.php` (verification and audit)

```
verify_csrf();
$action   = (string) ($_POST['action'] ?? '');
$entityId = filter_input(INPUT_POST, 'entity_id', FILTER_VALIDATE_INT);

if ($action === 'verify_user') {
    db()->prepare('UPDATE users SET verified_at = NOW()
                   WHERE id = ? AND role = "landlord"')->execute([$entityId]);
    audit('admin.user_verified', 'user', $entityId);
    flash('success', 'Landlord verified.');
}
```

`AND role = "landlord"` in the UPDATE is a second line of defence: even if a non-landlord identifier were submitted, no row would match. Narrowing the write at query level is more reliable than checking first and updating afterwards.

The audit trail survives deletion of the acting user — the foreign key uses `ON DELETE SET NULL` rather than cascade — because a record of who approved what must outlive the account.

Module 6 — `database/schema.sql` (data layer)

Seven tables in Third Normal Form, with integrity enforced by the database rather than by application code: eight foreign keys with deletion behaviour chosen per relationship, ENUM domains, CHECK constraints on monetary values, and composite indexes aligned to actual query patterns.

```
CREATE TABLE escrow_transactions (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    booking_id  BIGINT UNSIGNED NOT NULL UNIQUE,     -- UNIQUE enforces one escrow per booking
    amount      DECIMAL(10,2) NOT NULL,              -- never FLOAT for money
    status      ENUM('not_paid','held','released','refunded') NOT NULL DEFAULT 'not_paid',
    reference_code VARCHAR(40) NULL UNIQUE,
    CONSTRAINT fk_escrow_booking FOREIGN KEY (booking_id)
        REFERENCES bookings(id) ON DELETE CASCADE,
    CONSTRAINT chk_escrow_amount CHECK (amount >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

The `UNIQUE` on `booking_id` is what makes the one-to-one relationship real. Without it the relationship would exist only as an intention in application code, and a second escrow row for the same booking would be accepted.

5.2.4 Security implementation

Table 5.2.1 — Security controls and where they are verified

Control	Implementation	Verified by
SQL injection prevention	PDO prepared statements, native prepares	TC-31 — seven payloads treated as literal text
Password storage	`password_hash()` / `password_verify()`	TC-21 — hash asserted in the database
CSRF protection	Per-session token, `hash_equals()` constant-time comparison	TC-12 — control holds; DEF-012 on the status code
Session hardening	HttpOnly, SameSite=Lax, ID rotation on sign-in	TC-28, TC-35
Output escaping	`e()` on every echoed variable	TC-29 — injected payload rendered as text
Authorisation	Server-side role and ownership checks	TC-14, TC-33
Upload safety	Content inspection, 5 MB limit, randomised stored name	TC-19, TC-34
Brute-force resistance	Session-based throttle, 429 after five failures	TC-11
Response headers	CSP, X-Frame-Options, nosniff, Referrer-Policy	TC-28

5.2.5 Known limitations in the delivered code

Four defects were identified during testing and remain open. They are detailed in section 5.4.6; they are noted here because they are properties of the code rather than of the tests.

Table 5.2.2 — Open defects

ID	Severity	Description	Fix
DEF-011	Critical	Uploaded documents written to `public/uploads` and served directly by Apache with no authorisation check	Store outside the web root; stream through an authenticated handler
DEF-010	High	`available_rooms` is displayed but never read or decremented by `book.php`	Availability check and decrement inside the existing transaction
DEF-012	Medium	`verify_csrf()` returns HTTP 419, which Apache rewrites to 500	Return 403 Forbidden
DEF-013	Low	`label { display: block }` overrides the `[hidden]` attribute	Add `[hidden] { display: none !important; }`

5.3 Testing

This section documents the verification of EMERGEX: what was tested, how, with which tools, and on what basis a build is judged fit to demonstrate. Section 5.4 presents the test cases themselves, the results of executing them, and the defects that execution uncovered.

Because no real money moves, the correctness of the *state machine* around money is what matters: a deposit recorded as released when it was never held would be invisible to a user but wrong in the data. Several tests therefore assert on database rows rather than on screen text.

5.3.1 Objectives

1. Verify that every implemented feature behaves as specified.
2. Verify that data written by the application is correct and complete in the database, not merely plausible on screen.
3. Verify that access controls, input handling and session management resist ordinary hostile input.
4. Establish an automated regression suite that can be re-run on demand, so alater change cannot silently undo an earlier fix.
5. Record any defect found, with enough detail to reproduce and fix it.

5.3.2 Scope

In scope

- All public pages: home, search, listing detail, how-it-works, privacy, sign-in, registration
- All three authenticated roles
- The complete booking lifecycle: instant book, request-and-approve, request-and-decline, deposit simulation, check-in confirmation
- NSFAS document upload and administrator review
- Landlord verification and the audit trail
- Security controls: authentication, authorisation, CSRF, session handling, input escaping, upload handling, error disclosure

Out of scope

- **Load and performance testing.** The prototype runs on a single developer machine against seeded data; a throughput figure measured there would not be meaningful.
- **Cross-browser testing.** Execution used Chromium only.
- **Real payment integration.** None exists to test.
- **Penetration testing.** The security layer verifies that specific controls hold; it is not an adversarial assessment.

5.3.3 Test strategy — the six levels

Testing is organised into six levels. Each answers a different question, and a defect that escapes one level should be caught by the next.

Table 5.3.1 — Test levels

Level	Question it answers	Typical assertion	Tests
UI	Does the interface render correctly and accessibly?	The verified badge is visible; the grid stacks on mobile	25
Functional	Does the application follow the rules?	A past move-in date is refused; a student cannot open admin.php	39
Integration	Does what appears on screen reach MySQL?	The escrow row exists with status `held` and the correct amount	9
End-to-end	Do complete journeys work across roles?	Student books, landlord approves, deposit completes	5
Regression	Has a previous fix been undone?	Security headers still present; prices still formatted in Rand	24
Security	Do the controls hold under hostile input?	An injection payload is treated as literal text	16

Ten tests carry two level tags. A test proving a student cannot reach the administrator dashboard is simultaneously a functional rule and an access control, and is counted in both filters without being duplicated in code.

Relationship to earlier phases

Test design follows the V-model: each development phase has a corresponding test level, and the tests for a phase are designed while that phase is specified rather than afterwards.

Table 5.3.2 — V-model mapping

Development phase	Test level	Suite
3.6 Functional Requirements	Acceptance testing	End-to-end, Functional
3.7 Non-Functional Requirements	Security, usability, accessibility	Security, UI
4.3 Architectural Design	Integration testing	Integration
4.5 Database Design	Integration testing	Integration
4.7 Interface Design	Interface testing	UI
5.2 Coding	Component testing	Functional

5.3.4 Test environment

Table 5.3.3 — Environment specification

Component	Specification
Operating system	Windows 11
Web server	Apache (XAMPP), `localhost`
Application runtime	PHP 8, served from `application/public`
Database	MySQL 8, schema `emergex`, port 3306
Browser under test	Chromium (Playwright-managed build), headless
Test runtime	.NET 9.0
Test data	Three demo accounts and four published properties,loaded by `database/install.php`

The installer drops and recreates every table before seeding, so the suite can be returned to a known state at any time. This matters because several tests create bookings, register users and upload documents; without a deterministic starting point the assertions on counts would drift between runs.

5.3.5 Tools

Table 5.3.4 — Tooling

Tool	Version	Purpose
Playwright for .NET	1.62.0	Browser automation — drives a real Chromium instance
NUnit	4.6.1	Test framework: fixtures, categories, parameterised cases
MySqlConnector	2.4.0	Direct database access for the integrationlayer
Visual Studio 2022	—	Authoring and interactive execution via Test Explorer
.NET CLI	9.0	Recorded runs producing TRX and HTML artefacts
BYB Testing Tool	—	Purpose-built test case management tool used to author and track the 36 cases

Playwright was chosen over Selenium for two reasons. Its assertions are *web-first*: `Expect(locator).ToBeVisibleAsync()` retries until a timeout rather than failing the instant an element is missing, which removes the explicit waits and arbitrary sleeps that make Selenium suites flaky. Second, it provides an isolated browser context per test at negligible cost, so no sessionleaks between tests.

5.3.6 Framework architecture

The suite is not a flat collection of scripts. Shared machinery lives in a `Support` namespace — the test harness — separate from the tests themselves.

Table 5.3.5 — Framework components

Component	Responsibility
`TestConfig`	Base URL, demo credentials and seeded-data constants in one place. Every value is overridable by environment variable so the suite runs on another machine without code changes.
`BaseTest`	Base class for every test. Provides a fresh browser context, sign-in and sign-out helpers, and a helper that POSTs directly to a handler with a valid CSRF token.
`Database`	Parameterised MySQL access used by the integration and security layers, plus cleanup helpers.
`TestFiles`	Upload fixtures built in memory — a minimal valid PDF, a 1×1 PNG, and an inert PHP file used to test disguised uploads
`Pages/`	Page objects. **Every CSS selector in the suite lives here and nowhere else**

> **Why the Page Object Model matters.** Tests express intent — *open the listing, start the booking, submit it* — while page objects hold the knowledge of which DOM element that means. When the markup changes, the fix is one line in one file rather than a search across ninety tests. It also makes tests readable to someone who has never seen the HTML.

5.3.7 Test design principles

- **No selector appears in a test.**
- **No fixed waits.** There is not one `Thread.Sleep` in the suite.
- **Every test cleans up after itself**, so the suite is repeatable without reseeding between runs.
- **Negative cases are first-class.** Roughly half the suite asserts that something is correctly refused. A suite that only proves the happy path proves very little.
- **Integration means the database.** Integration tests assert on MySQL rows, the only way to prove persistence actually happened.
- **Tests that document defects are kept.** Four tests assert correct behaviour the build does not implement and therefore fail. Deleting them would raise the pass rate and hide four real defects.

5.3.8 Entry and exit criteria

Table 5.3.6 — Entry and exit criteria

Entry criteria	Exit criteria
• Application deployed and reachable • Database schema created and seeded • Test project compiles without error • Browser binaries installed	• All 36 test cases executed • Every failure traced to a defect or a test fault • No unexplained failure remaining • Results recorded to file, not only to screen

5.3.9 Executing the suite

Two execution paths were used. **Visual Studio Test Explorer** during development: grouping by trait presents the six levels as folders and individual tests can be re-run in isolation. **The .NET CLI** for the recorded run, because Test Explorer holds results in memory and does not honour the logger configuration, so it cannot produce a results file.

```
dotnet test --settings .runsettings
```

This writes a TRX result file and a browsable HTML report to `TestResults/`. Individual levels can be run with `--filter "Category=Security"` and equivalents.

5.4 System Testing

5.4.1 Test case design

Thirty-six test cases were designed, each specifying preconditions, numbered steps, and an expected result. They are listed in summary in section 5.4.4 and in full in Appendix A.

Cases were selected by risk rather than by feature list, using four criteria in order: **impact if it fails** (deposits and identity documents rank highest), **frequency of use** (search and booking affect every user), **likelihood of breaking** (validation, filters and access control are conditional logic, where defects cluster), and **invisibility** (a missing security header fails silently, so automation is the only way it is caught).

An earlier set of thirty cases was discarded during design. Nine of those thirty described functionality the implemented system does not contain — creating a property listing, hiding a listing as an administrator, a "fully booked" overlay, a payment-processing loading state, webhook payment synchronisation, a declined-payment path, rejecting a landlord's verification, editing a landlord profile, and validating a rent-price field. Retaining them would have produced a report full of untestable cases. They were replaced with cases covering behaviour the system does implement but which the original set had not considered: CSRF enforcement, login throttling, security response headers, session cookie hardening, output escaping, audit logging, randomised upload filenames, and price tampering through request parameters.

> **On the ratio of cases to test methods.** Thirty-six test cases are implemented by 118 automated test executions. This is expected rather than inconsistent. A case whose expected result lists four conditions requires four assertions, and a case specifying several inputs — four injection payloads, six public pages — becomes several parameterised executions of one method. The test case is the unit of specification; the test method is the unit of execution.

5.4.2 Case status and execution result

Two distinct pieces of state are tracked for every test case, and conflating them is a common source of confusion.

Table 5.4.1 — Case status versus execution result

Concept	What it records	Values used
Case status	The lifecycle of the specification — has this case been written, reviewed and agreed?	`draft` → `ready` → `approved`
Execution result	What happened the last time the case was run	`passed`, `failed`, `blocked`, `not run`

A case can be approved and still fail: approval means the specification is correct, not that the application satisfies it. Keeping the two separate also preserves history — the result belongs to a particular run, so a case that passes today and fails after tomorrow's change has two results, not one overwritten value.

All 36 cases carry the status **approved**.

5.4.3 Test run record

Table 5.4.2 — Test run TR-001

Field	Value
Run identifier	TR-001
Date executed	6 September 2026
Executed by	Novithan Naidoo (Group 55)
Build under test	EMERGEX prototype, local XAMPP deployment
Environment	Windows 11 · Apache · PHP 8 · MySQL 8 · Chromium (headless)
Preconditions	Database dropped and reseeded via `install.php` immediately before execution
Scope	All 36 test cases, all six levels
Command	`dotnet test --settings .runsettings`
Evidence	`TestResults/*.trx` and `TestResults/emergex-test-report html`

A **test run** is a single, dated execution of a defined set of test cases against one build in one environment. It is the unit that carries results: the cases are reusable, the run is the record of what they proved on a given day. Re-running the suite after a fix produces TR-002, leaving TR-001 intact as evidence of the state before.

5.4.4 The 36 test cases and their outcomes

Table 5.4.3 — Test case inventory (all cases approved)

ID	Title	Level	Priority	Result (TR-001)
TC-01	Verified Landlord Trust Badge Rendering	UI	Medium	Pass
TC-02	NSFAS Accreditation Badge Visibility	UI	Medium	Pass
TC-03	Booking Call-to-Action Reflects Booking Mode	UI	High	Pass
TC-04	Search Grid Mobile Responsiveness	UI	Medium	Pass
TC-05	Role-Aware Header Navigation	UI	Medium	Pass
TC-06	Accessible Empty State and Status Messaging	UI	Low	Pass
TC-07	Register New Student Account	Functional	High	Fail — DEF-013
TC-08	Enforce Password Policy on Registration	Functional	High	Pass
TC-09	Reject Duplicate Email Registration	Functional	High	Pass
TC-10	Reject Invalid Login Credentials	Functional	High	Pass
TC-11	Throttle Repeated Failed Login Attempts	Functional	High	Pass
TC-12	Reject Form Submission Without CSRF Token	Functional	High	Partial — DEF-012
TC-13	Student Logout and Session Termination	Functional	High	Pass
TC-14	Block Unauthorised URL Access by Role	Functional	High	Pass
TC-15	Filter Properties by NSFAS Status	Functional	High	Pass
TC-16	Filter Properties by Maximum Rent	Functional	High	Pass
TC-17	Keyword Search and Empty Result Handling	Functional	Medium	Pass
TC-18	Reject Invalid Booking Submission	Functional	High	Pass
TC-19	Reject Malicious File Uploads	Functional	High	Pass
TC-20	Search Grid Matches Database Contents	Integration	High	Pass
TC-21	Registration Stores Hashed Password and Role	Integration	High	Pass
TC-22	Instant Booking Persists Booking and Escrow Records	Integration	High	Pass
TC-23	Escrow Simulation Transitions Through Every State	Integration	High	Pass
TC-24	Document Upload and Audit Log Persistence	Integration	High	Pass
TC-25	Complete Instant Book and Simulated Payment Journey	E2E	High	Pass
TC-26	Request, Landlord Decision and Deposit Journey	E2E	High	Pass
TC-27	Landlord Verification and Document Review Journey	E2E	High	Pass

TC-28	Security Headers and Session Cookie Hardening	Regression	High	**Pass**
TC-29	Output Escaping and Currency Formatting	Regression	High	**Pass**
TC-30	Prevent Price Tampering via Request Parameters	Regression	High	**Pass**
TC-31	Resist SQL Injection on Public Inputs	Security	High	**Pass**
TC-32	Prevent Account Enumeration and Credential Disclosure	Security	High	**Pass**
TC-33	Enforce Access Control on Documents and Bookings	Security	High	**Fail — DEF-011**
TC-34	Harden File Upload Handling	Security	High	**Pass**
TC-35	Protect Session Transport and Prevent Caching	Security	High	**Pass**
TC-36	Suppress Internal Detail in Error Responses	Security	Medium	**Pass**

5.4.5 Execution results

Table 5.4.4 — Execution summary

Measure	Value
Test cases executed	36 of 36 (100%)
Test cases passed	33
Test cases not passed	3 (TC-07, TC-12, TC-33)
Test methods executed	118
Passed	114
Failed	4
Skipped	0
Pass rate	96.6%
Duration	46.5 seconds
Failures traced to a defect	4 of 4 (100%)
Unexplained failures	0

Table 5.4.5 — Results by test level

Level	Executed	Passed	Failed	Failing test
UI	25	25	0	—
Functional	39	38	1	CSRF rejection status (DEF-012)
Integration	9	9	0	—
End-to-end	5	5	0	—
Regression	24	22	2	Room availability (DEF-010); student-number visibility (DEF-013)
Security	16	15	1	Document access control (DEF-011)

Level totals exceed 118 because ten tests are tagged with two levels.

Four test *methods* failed but only three test *cases* did. The fourth failure belongs to DEF-010, the missing room-availability check, which has no corresponding test case: the control it would verify does not exist in the application, so there was nothing to specify. It is asserted by a test in the regression suite and recorded in the defect register, but it is not counted against any case.

5.4.6 Defects identified

Four defects were identified. All four were found by automated tests; none was visible through ordinary manual use of the application, which is the central argument for the effort invested in the suite.

Table 5.4.6 — Defect register

ID	Severity	Description	Recommended fix
DEF-011	Critical	Uploaded identity documents are written to `public/uploads` and served directly by Apache with no authorisation check. An anonymous request for a document URL returned HTTP 200 and the file contents. The stored filename is forty random hexadecimal characters and cannot be guessed, but an unguessable URL is obscurity, not access control: anyone who obtains the link through a shared screenshot, a Referer header or browser history synchronisation can read another student's identity document without a session.	Store uploads outside the web root and serve them through an authenticated PHP handler that checks `user_documents.user_id` against the current session before streaming bytes.
DEF-010	High	`book.php` never reads `available_rooms`, and no code path decrements it. A property advertising zero available rooms accepts an unlimited number of bookings, while the landlord's dashboard continues to display zero. The column is declared `SMALLINT UNSIGNED`, so the database cannot store a negative value — but that is a storage constraint, not a business rule, and it is never exercised because nothing writes to the column.	Add an availability check inside the existing transaction in `book.php`, and decrement the column on successful booking.
DEF-012	Medium	`verify_csrf()` sets HTTP 419. That code is a framework convention and is not registered withIANA, so Apache does not recognise the status line and substitutes 500 Internal Server Error. The CSRF protection itself is effective — the request is refused and no session is issued — but every legitimately blocked submission is reported to the user and to any monitoring as a server crash, and the intended "The form expired" guidance never reaches the page.	Return 403 Forbidden from `verify_csrf()` in `bootstrap.php`.
DEF-013	Low	`app.js` correctly sets the `hidden` property on the `.student-only` label when the Landlord role is selected. However `app.css` declares `label { display: block }`, and an author stylesheet rule takes precedence over the user-agent rule `[hidden] { display: none }`. The attribute is therefore set while the field remains visible, so a landlord is shown — and can complete — a student-number field that does not apply to them.	Add `[hidden] { display: none !important; }` to `app.css`.

5.4.7 Evaluation of the testing results

Overall assessment

The application is functionally sound. All twenty-seven functional, integration and end-to-end cases passed, including every booking journey and every escrow state transition. The core value proposition —

that a student can find verified accommodation, book it, and move a deposit through a transparent escrow model — works correctly and persists correct data at every step.

Security is largely well implemented, which is worth stating plainly because failures are easier to notice than passes. Database queries are parameterised throughout: seven distinct SQL injection payloads across the search and sign-in interfaces were treated as literal text, with no error disclosure and no authentication bypass. Passwords are hashed. Session identifiers are regenerated on sign-in, preventing session fixation. The session cookie is HttpOnly and SameSite=Lax and is not readable from page script. Security response headers are present. Uploads are validated by content inspection rather than by file extension, and a PHP file declaring an image MIME type was correctly rejected. Output is escaped, so a stored HTML payload rendered as literal text rather than executing.

The significant finding

DEF-011 is the defect that matters. The system's purpose is to protect students from accommodation fraud, and it asks them to upload NSFAS documentation — which for a South African student typically carries an identity number, a name, and financial information. Storing those files in a publicly served directory undermines the trust proposition the platform is built on. The random filename means the exposure requires the URL to leak rather than being enumerable, which lowers the likelihood but not the impact. It is recommended that this be resolved before any demonstration involving real documents.

What automation contributed

None of the four defects would plausibly have been found by manual testing. A tester does not delete a hidden CSRF field before submitting a form; does not copy a stored filename out of the database and request it in a private window; does not set a property's room count to zero and then attempt to book it; and would very likely accept the student-number field as correct, because the script governing it is written correctly — only the rendered result is wrong. The value of the suite lies less in confirming that the happy path works than in exercising the paths nobody thinks to try.

A methodological finding

> The SQL injection test initially appeared to pass when driven through the user interface. Investigation showed that the sign-in email field is declared `type="email" required`, so the browser refused to submit the malformed payload and the request never reached the server. The test was proving that Chromium validates email addresses, not that the application resists injection.
>
> The test was rewritten to POST directly, bypassing client-side validation, at which point it exercised the PHP and passed on its merits. The general lesson is that **client-side validation is a usability feature and never a security control**, because the attacker chooses the client. Security testing conducted exclusively through the interface will report false passes.

5.4.8 Limitations

- Execution was against Chromium only. Rendering differences in Firefox or Safari would not be detected.
- The suite runs against seeded data of four properties and three users. Behaviour under realistic data volumes — pagination, query performance, index effectiveness — is untested.

- No load or concurrency testing was performed. The booking transaction has not been tested for race conditions under simultaneous requests, which is relevant given DEF-010.
- Accessibility testing was structural (alternative text, ARIA roles, focus order). No assistive-technology or colour-contrast audit was conducted.

5.4.9 Recommendations

1. **Resolve DEF-011 before any demonstration using real documents.** Move uploads outside the web root and serve them through an authenticated handler.
2. **Implement the room-availability control (DEF-010)** inside the existing booking transaction, and add the corresponding integrity constraint to the data model.
3. **Correct the CSRF rejection status (DEF-012)** to 403, so blocked submissions are distinguishable from genuine server errors in logs.
4. **Correct the hidden-field rendering (DEF-013)** with an explicit `[hidden]` rule in the stylesheet.
5. **Re-run the suite after each fix.** The four failing tests are the acceptance criteria for those fixes; each should turn green without any other test turning red.
6. **Extend the suite as features are added.** Nine cases were removed because their features do not exist. If listing creation or administrative moderation is implemented, the corresponding cases should be restored.

5.5 Installation

5.5.1 Requirements

Table 5.5.1 — Prerequisites

Component	Requirement
Operating system	Windows 10 or 11 (the procedure also works on Linux and macOS with equivalent paths)
Web server and runtime	XAMPP with Apache and PHP 8.1 or later
Database	MySQL 8 or MariaDB, included with XAMPP
PHP extensions	pdo, `pdo_mysql`, `fileinfo` — all enabled by default in XAMPP
Browser	Any current browser
Disk space	Approximately 50 MB for the application and seeded database

`fileinfo` is worth checking specifically: upload validation inspects file contents through it, and without the extension the upload feature cannot determine a file's true type.

5.5.2 Installation procedure

1. **Install XAMPP** to `C:\xampp`.
2. **Copy the application folder** into the web root, so that it sits at `C:\xampp\htdocs\emergex`.
3. **Start Apache and MySQL** from the XAMPP Control Panel and confirm both show as running.
4. **Check the configuration** in `app/config/config.php`. The supplied values match a standard local XAMPP installation. **If the folder name or web path differs, `base_url` must be updated to match** — see 5.5.5.
5. **Create and seed the database:**

   ```
   C:\xampp\php\php.exe C:\xampp\htdocs\emergex\database\install.php
   ```

   The installer creates the schema, loads demonstration data and prints the demonstration accounts. It must be run from the command line — it refuses to execute over HTTP, so a deployed system cannot be reset by a visitor.
6. **Open the application** at `http://localhost/emergex/public/`.

5.5.3 Demonstration accounts

Table 5.5.2 — Seeded accounts

Role	Email	Password
Student	`student@emergex test`	`Student@55`
Landlord	`landlord@emergex.test`	`Landlord@55`
Administrator	`admin@emergex.test`	`Admin@55Secure`

These credentials are for local assessment only. The passwords must not be reused, and the seeded database must not be exposed publicly.

5.5.4 Verifying the installation

Each step below exercises a different layer, so a failure identifies where the problem is.

Table 5.5.3 — Verification steps

#	Action	Expected result
1	Open the landing page	Page renders with styling and images — confirms Apache and the asset paths
2	Open the property search	Four seeded listings appear — confirms the database connection and seeding
3	Sign in as the student	Lands on the student dashboard — confirms sessions and password verification
4	Open an instant-book listing and complete a booking	Booking appears as approved — confirms the write path and the transaction
5	Select *Simulate deposit hold*, then *Confirm check-in*	Status reaches *completed* with a reference beginning `SIM-` — confirms the state machine
6	Sign in as the landlord	The seeded pending request is listed — confirms role routing
7	Sign in as the administrator and open Reports	Report centre loads and a CSV downloads — confirms the reporting module

Two supplied checks can also be run:

```
C:\xampp\php\php.exe tests\security_checks.php      # static security checks
C:\xampp\php\php.exe tests\acceptance_tests.php     # PHP acceptance suite
```

The automated suite described in 5.3 and 5.4 is a separate project with its own installation procedure — see Appendix B.

5.5.5 Troubleshooting

Table 5.5.4 — Common problems

Symptom	Cause	Resolution
Pages load but every link gives "Not Found"	`base_url` does not match the actual folder	Edit `app/config/config.php` so `base_url` matches the path under `htdocs`. This is the single most common setup failure and is easy to miss, because the first page loads correctly.
`'php' is not recognised`	PHP is not on the system PATH	Use the full path `C:\xampp\php\php.exe`, or add `C:\xampp\php` to PATH
Installer prints "Run this installer from the command line only"	It was opened in a browser	Run it from Command Prompt. This restriction is deliberate
MySQL will not start in XAMPP	Port 3306 is in use by another MySQL installation	Stop the other service, or change the port in both the XAMPP configuration and `config php`
Search returns no listings	The installer has not been run, or seeding failed	Re-run `install.php` and check its output for errors
Uploads are rejected regardless of file type	The `fileinfo` extension is disabled	Enable `extension=fileinfo` in `php.ini` and restart Apache

5.5.6 Resetting and backing up

Reset. Re-running `install.php` drops and recreates every table before reseeding. This is the quickest way to return a demonstration to a known state. It destroys all data, so anything worth keeping must be backed up first.

Backup:

```
C:\xampp\mysql\bin\mysqldump.exe -u root emergex > emergex_backup.sql
```

Restore:

```
C:\xampp\mysql\bin\mysql.exe -u root emergex < emergex_backup.sql
```

Take a backup before any demonstration. Restoring a dump takes seconds; rebuilding a demonstration state by hand during an assessment does not.

5.5.7 Deployment considerations

The procedure above describes a local assessment environment. A production deployment would additionally require: HTTPS, so that the `secure` session cookie flag takes effect; a database user with only the privileges the application needs, rather than `root`; the document storage weakness (DEF-011)

resolved; a web server configured to expose only the `public` directory; and error display disabled with logging to file.

None of these is in scope for the prototype, but all are prerequisites before the system could handle real student data.

Conclusion

Phase 5 delivered a working student accommodation platform together with the evidence that it works. The application implements verified listings, two booking paths, a simulated escrow lifecycle, document review and an audit trail across three user roles, built on a normalised MySQL schema with integrity enforced by the database.

Thirty-six test cases were designed from the implemented behaviour of the system and automated as 118 executable tests across six levels. All thirty-six were executed in a single recorded run. One hundred and fourteen tests passed, four failed, and every failure was traced to a documented defect rather than to a fault in the tests.

The system is functionally correct and, with one significant exception, securely implemented. The four defects identified are individually addressable, and the tests that revealed them serve as the acceptance criteria for their resolution. The suite is repeatable, self-cleaning and version-controlled alongside the application, so it will continue to protect these findings as development continues.

Appendix A — Test case specifications

Each case lists its preconditions, numbered steps and expected result. Source of record: `TestCases/test_cases_v2.json`.

A.1 User interface

TC-01 — Verified Landlord Trust Badge Rendering

Level: UI · *Priority:* Medium · *Result:* Pass
Precondition: Demo data loaded; "Emerald House Hatfield" belongs to the verified demo landlord.

1. Open properties.php as an anonymous visitor.
2. Open the listing "Emerald House Hatfield" from the results grid.
3. Inspect the landlord information panel.

Expected: The verified badge is visible and reads "Verified landlord". The badge is absent for listings whose landlord has a null verified_at.

TC-02 — NSFAS Accreditation Badge Visibility

Level: UI · *Priority:* Medium · *Result:* Pass
Precondition: Three of four seeded listings are NSFAS accredited; "Durban North Student Lofts" is not.

1. Open the detail page for "Emerald House Hatfield" (accredited).
2. Confirm the NSFAS badge is present.
3. Open the detail page for "Durban North Student Lofts" (not accredited).
4. Confirm the NSFAS badge is absent.

Expected: The badge renders for accredited listings and not for non-accredited listings.

TC-03 — Booking Call-to-Action Reflects Booking Mode

Level: UI · *Priority:* High · *Result:* Pass

1. Open an instant-book listing and read the primary button label.
2. Open a request-flow listing and read the primary button label.
3. Confirm the sandbox payment disclaimer is shown on both.

Expected: Labels are "Instant Book" and "Request booking" respectively. Both pages display the "No real payment is taken" disclaimer.

TC-04 — Search Grid Mobile Responsiveness

Level: UI · *Priority:* Medium · *Result:* Pass

1. Set the viewport to 375 × 812.
2. Open properties.php.
3. Compare the bounding boxes of the first two property cards.
4. Compare document scroll width against viewport width.
5. Open the mobile navigation toggle.

Expected: Cards are vertically stacked at the same x-offset. Document scroll width does not exceed viewport width. The toggle is visible, has an accessible name, and reveals the menu.

TC-05 — Role-Aware Header Navigation

Level: UI · *Priority:* Medium · *Result:* Pass

1. Open the site while signed out and inspect the header.
2. Sign in as the demo student and inspect the header.
3. Confirm the Reports link is not present.
4. Sign out, sign in as administrator, confirm the Reports link is present.

Expected: Header links match the current role in all three states. Reports appears for the administrator only.

TC-06 — Accessible Empty State and Status Messaging

Level: UI · *Priority:* Low · *Result:* Pass

1. Search for a term matching no listing.
2. Confirm the empty state renders with guidance copy.
3. Sign in and inspect the welcome flash message for role="status".
4. Inspect the alt attribute of each property card image.

Expected: The empty state is visible with helpful copy. The flash element carries role="status". Every property image has a non-empty alt attribute that is not merely the filename.

A.2 Functional

TC-07 — Register New Student Account

Level: Functional · *Priority:* High · *Result:* Fail (DEF-013)

1. Open register.php.
2. Select the Student role and confirm the student number field becomes required.
3. Complete all required fields with valid data and accept the terms.
4. Submit the form.

Expected: The account is created, the user is signed in, and the browser lands on dashboard.php with a success message. Selecting Landlord hides the student-number field.

TC-08 — Enforce Password Policy on Registration

Level: Functional · *Priority:* High · *Result:* Pass

1. Open register.php and select the Student role.
2. Complete the form using a weak password such as "pass".
3. Submit the form.
4. Query the users table for the submitted email address.

Expected: The form is redisplayed with a password policy error and no user row was created.

TC-09 — Reject Duplicate Email Registration

Level: Functional · *Priority:* High · *Result:* Pass

1. Open register.php.

2. Complete the form using the existing demo student email address.
3. Submit the form.

Expected: Registration is refused with a clear message and no duplicate user row is created.

TC-10 — Reject Invalid Login Credentials

Level: Functional · *Priority:* High · *Result:* Pass

1. Open login.php.
2. Submit a valid email with an incorrect password.
3. Submit an unregistered email address.
4. Confirm the error wording is identical in both cases.

Expected: Both attempts are refused. The message is generic, preventing account enumeration.

TC-11 — Throttle Repeated Failed Login Attempts

Level: Functional · *Priority:* High · *Result:* Pass

1. Open login.php.
2. Submit incorrect credentials five times in succession.
3. Submit a sixth attempt and observe the response.

Expected: The sixth attempt is refused with HTTP 429 and a throttling message rather than a normal credential error.

TC-12 — Reject Form Submission Without CSRF Token

Level: Functional · *Priority:* High · *Result:* Partial (DEF-012)

1. Open login.php.
2. Remove the hidden CSRF token field from the form via the DOM.
3. Submit the form with otherwise valid credentials.
4. Observe the HTTP response status.

Expected: The request is refused with a clean 4xx status and the user is not signed in.
Actual: Refusal occurs correctly, but the status is 500 rather than the intended 419.

TC-13 — Student Logout and Session Termination

Level: Functional · *Priority:* High · *Result:* Pass

1. Sign in as the demo student and confirm dashboard.php loads.
2. Sign out using the header link.
3. Navigate directly to dashboard.php.
4. Confirm the session cookie value has changed or been cleared.

Expected: After sign-out, dashboard.php redirects to login.php and the previous session identifier is no longer valid.

TC-14 — Block Unauthorised URL Access by Role

Level: Functional · *Priority:* High · *Result:* Pass

1. Signed in as a student, navigate directly to admin.php andlandlord.php.
2. Sign out and navigate directly to dashboard.php.
3. Signed in as a landlord, attempt to submit a booking.

4. Signed in as one student, attempt to simulate a deposit on another user's booking.

Expected: Cross-role access returns HTTP 403. Anonymous access redirects to login.php. The landlord booking attempt is forbidden. The foreign booking action returns HTTP 404 rather than confirming the booking exists.

TC-15 — Filter Properties by NSFAS Status

Level: Functional · *Priority:* High · *Result:* Pass

1. Open properties.php and note the unfiltered result count.
2. Apply the NSFAS filter and count the results.
3. Confirm the non-accredited listing is absent.
4. Apply the NSFAS and Instant Book filters together.

Expected: Unfiltered returns 4. NSFAS alone returns 3. The combined filter returns the intersection, not the union.

TC-16 — Filter Properties by Maximum Rent

Level: Functional · *Priority:* High · *Result:* Pass

1. Open properties.php.
2. Set the maximum rent filter to 5000 and apply it.
3. Read the rent shown on each returned card.
4. Clear the filters and confirm all listings return.

Expected: Two listings are returned, both at or below R5 000. Clearing filters restores all four.

TC-17 — Keyword Search and Empty Result Handling

Level: Functional · *Priority:* Medium · *Result:* Pass

1. Search for a city name and confirm the matching listing is returned.
2. Search for a nearby institution and confirm the matching listing is returned.
3. Search for a nonsense term with no matches.
4. Observe the response.

Expected: Relevant listings are returned for valid terms. The nonsense search returns zero cards, an empty state with guidance copy, and HTTP 200 — not a server error.

TC-18 — Reject Invalid Booking Submission

Level: Functional · *Priority:* High · *Result:* Pass

1. Open the booking form for a valid listing.
2. Submit with a move-in date in the past.
3. Submit with a message exceeding the permitted length.
4. Request the booking form for a property id that does not exist.

Expected: The first two submissions are refused with validation errors. The unknown property returns HTTP 404. No bookings row is created by any of the three.

TC-19 — Reject Malicious File Uploads

Level: Functional · *Priority:* High · *Result:* Pass

1. Open the student dashboard document upload control.
2. Upload a file containing PHP source but named with a .png extension and an image MIME type.

3. Observe the response and check the uploads directory.
4. Upload a genuine PDF and confirm it is accepted.

Expected: The disguised file is rejected and no row or stored file is created. The genuine PDF is accepted and recorded with status "pending".

A.3 Integration

TC-20 — Search Grid Matches Database Contents

Level: Integration · *Priority:* High · *Result:* Pass

1. Query the properties table directly for published rows.
2. Open properties.php and read the rendered card titles.
3. Compare the two sets.
4. Set one property's status to draft, reload, and compare again.

Expected: The rendered set matches the queried set exactly. After the status change the drafted listing disappears and the count decreases by one.

TC-21 — Registration Stores Hashed Password and Correct Role

Level: Integration · *Priority:* High · *Result:* Pass

1. Register a new student account through the web form.
2. Query the users table for the created row.
3. Inspect the password_hash column.
4. Inspect the role and status columns.

Expected: The row exists with role "student" and status "active". The password_hash column does not contain the submitted password and matches a recognised hash prefix.

TC-22 — Instant Booking Persists Booking and Escrow Records

Level: Integration · *Priority:* High · *Result:* Pass

1. Submit an instant booking through the web form.
2. Query the bookings table for the newest row for that student.
3. Query escrow_transactions for the matching booking id.
4. Compare the escrow amount against the property's deposit amount.

Expected: One booking row with status "approved" and correct ids. One escrow row with status "not_paid" and an amount equal to the property's stored deposit.

TC-23 — Escrow Simulation Transitions Through Every State

Level: Integration · *Priority:* High · *Result:* Pass

1. Confirm the escrow row starts at status "not_paid".
2. Trigger the deposit simulation and re-query.
3. Confirm the dashboard reports the deposit as securely held.
4. Confirm check-in and re-query.
5. Confirm a payment reference is displayed.

Expected: Escrow status progresses not_paid → held → released. The booking reaches "completed". A reference beginning "SIM-" is displayed and no payment gateway is contacted.

TC-24 — Document Upload and Audit Log Persistence

Level: Integration · *Priority:* High · *Result:* Pass

1. Upload a valid PDF from the student dashboard.
2. Query user_documents and compare the stored path against the original filename.
3. Sign in as the administrator and verify a landlord.
4. Query the audit_logs table for the newest entry.

Expected: The stored filename differs from the original and is not guessable; the original name is retained separately. The audit log contains an "admin.user_verified" entry naming the acting administrator and the affected user.

A.4 End-to-end

TC-25 — Complete Instant Book and Simulated Payment Journey

Level: E2E · *Priority:* High · *Result:* Pass

1. Sign in as the demo student.
2. Search and filter the listings to instant-book only.
3. Open a listing from the grid and start the instant booking.
4. Submit the booking and confirm it is approved on the dashboard.
5. Simulate the deposit hold.
6. Confirm check-in and read the payment reference.

Expected: The booking is approved immediately, the deposit moves held → released, the booking reaches "completed", and a simulated reference is displayed. The escrow row agrees with the screen at every step.

TC-26 — Accommodation Request, Landlord Decision and Deposit Journey

Level: E2E · *Priority:* High · *Result:* Pass

1. As the student, submit a booking request with a message.
2. Confirm the dashboard shows "Awaiting landlord" and a response deadline.
3. Sign out and sign in as the landlord.
4. Approve the request and confirm the success message.
5. Sign back in as the student and simulate the deposit.
6. Repeat with a declined request and inspect the available actions.

Expected: The request starts pending with a 12-hour deadline. On approval the student sees "Approved", the deposit action unlocks, and escrow reaches "held". On decline the booking shows "Declined", no action buttons are offered, and escrow remains "not_paid".

TC-27 — Landlord Verification and Document Review Journey

Level: E2E · *Priority:* High · *Result:* Pass

1. Register a new landlord from the "List a property" call to action.
2. Confirm the dashboard reports verification as pending.
3. Sign in as the administrator and verify the new landlord.
4. Approve a student's uploaded NSFAS document.

5. Confirm both actions appear in the audit trail.
6. Sign in as each affected user and confirm the outcome is visible.

Expected: The landlord dashboard changes from "Verification pending" to "Identity verified". The student sees the document marked "Approved". Both actions are listed in the audit trail.

A.5 Regression

TC-28 — Security Headers and Session Cookie Hardening

Level: Regression · *Priority:* High · *Result:* Pass

1. Request any page and inspect the response headers.
2. Confirm X-Frame-Options, X-Content-Type-Options, Content-Security-Policy and Referrer-Policy are present and correctly valued.
3. Sign in and inspect the session cookie attributes.
4. Read document.cookie from page script and confirm the session cookie is absent.

Expected: X-Frame-Options is DENY, X-Content-Type-Options is nosniff, the CSP includes frame-ancestors 'none', and Referrer-Policy is set. The session cookie is HttpOnly with SameSite=Lax and is not readable from JavaScript.

TC-29 — Output Escaping and Currency Formatting

Level: Regression · *Priority:* High · *Result:* Pass

1. Store a property title containing an HTML injection payload.
2. Open that property's detail page.
3. Confirm the payload did not execute and is displayed as literal text.
4. Restore the original title.
5. Read every price on the search grid and check its format.

Expected: The injected script does not execute and renders as literal text. Every price matches "R" followed by space-separated thousands, for example "R 5 200".

TC-30 — Prevent Price Tampering via Request Parameters

Level: Regression · *Priority:* High · *Result:* Pass

1. Open the booking form for a listing whose deposit amount is known.
2. Inject additional monthly_rent and deposit_amount fields with altered values.
3. Submit the booking.
4. Query the escrow_transactions row created by the submission.
5. Compare its amount against the property's stored deposit amount.

Expected: The escrow amount equals the property's stored deposit, not the value supplied in the request.

A.6 Security

TC-31 — Resist SQL Injection on Public Inputs

Level: Security · *Priority:* High · *Result:* Pass · *OWASP:* A03 Injection

1. Submit the payload `' OR '1'='1` through the property search field.
2. Confirm the response is HTTP 200 with no database error text.

3. Confirm the search returns zero results rather than every listing.
4. Submit a stacked DROP TABLE payload and confirm the users table still exists.
5. Submit tautology payloads through the sign-in fields.

Expected: All payloads are treated as literal text. No SQLSTATE, syntax error or fatal error appears. A tautology does not act as a wildcard and does not authenticate. The schema is unchanged.

TC-32 — Prevent Account Enumeration and Credential Disclosure

Level: Security · *Priority:* High · *Result:* Pass · *OWASP:* A07

1. Attempt sign-in with a registered email and incorrect password; record the message.
2. Attempt sign-in with an unregistered email; record the message.
3. Compare the two messages.
4. Submit a registration that will fail, using a distinctive password.
5. Search the returned page source for that password.

Expected: Both failures produce an identical message. The rejected registration page source does not contain the submitted password.

TC-33 — Enforce Access Control on Documents and Bookings

Level: Security · *Priority:* High · *Result:* Fail (DEF-011) · *OWASP:* A01

1. Upload a document as the demo student and read its stored filename from the database.
2. Clear all cookies and request that file directly by URL.
3. Request the uploads directory itself and inspect the response.
4. Register a second student and POST a deposit action naming the first student's booking id.
5. Re-query the first student's escrow row.

Expected: The anonymous document request is refused. The uploads directory returns 403 or 404. The cross-account booking action returns HTTP 404 and the victim's escrow row is unchanged.
Actual: The cross-account and directory-listing controls hold. The anonymous document request returned HTTP 200 with the file contents — see DEF-011.

TC-34 — Harden File Upload Handling

Level: Security · *Priority:* High · *Result:* Pass · *OWASP:* A03 / A04

1. Upload a valid PDF whose filename contains directory traversal sequences.
2. Read the stored_name and original_name columns for that upload.
3. Upload a file containing PHP source but declaring a PDF MIME type.
4. Confirm the response and check whether a row was created.

Expected: The stored name is 40 random hex characters plus an extension derived from the sniffed MIME type, with no traversal sequences or path separators. The disguised PHP file is rejected and no row is created.

TC-35 — Protect Session Transport and Prevent Caching

Level: Security · *Priority:* High · *Result:* Pass · *OWASP:* A05

1. Record every request URL issued during a signed-in browsing session.
2. Search those URLs for a session identifier parameter.
3. Request the student dashboard and inspect its Cache-Control header.

Expected: No request URL contains a session identifier. The dashboard sends a Cache-Control header including no-store or no-cache.

TC-36 — Suppress Internal Detail in Error Responses

Level: Security · *Priority:* Medium · *Result:* Pass · *OWASP:* A05

1. Request a property detail page for an id that does not exist.
2. Request a property detail page with a non-numeric id.
3. Request a booking page with a negative property id.
4. Request a page that does not exist.
5. Inspect each response body for internal detail.

Expected: No response contains a server file path, "Stack trace", "SQLSTATE", or a PHP Warning, Notice or Deprecated message.

Appendix B — Reproducing the test run

The complete suite and its documentation are held in `EMERGEX.Tests/` alongside the application source.

B.1 Prerequisites

1. Apache and MySQL running under XAMPP.
2. .NET 9 SDK installed.
3. Playwright browser binaries installed:

   ```
   powershell -ExecutionPolicy Bypass -File bin\Debug\net9.0\playwright.ps1 install chromium
   ```

B.2 Procedure

1. Reset the database to a known state:

   ```
   C:\xampp\php\php.exe application\database\install.php
   ```

2. Execute the recorded run:

   ```
   dotnet test --settings .runsettings
   ```

3. Collect artefacts from `TestResults/`: the TRX result file and `emergex-test-report.html`.

Individual levels may be executed with `dotnet test --filter "Category=Security"` and the equivalents for UI, Functional, Integration, E2E and Regression.

B.3 Repository contents

Path	Contents
`UITests/`	Presentation and accessibility tests
`FunctionalTests/`	Authentication, search and booking behaviour
`IntegrationTests/`	Database persistence verification
`End-to-EndTests/`	Multi-role user journeys
`RegressionTests/`	Smoke pack, defect guards and data integrity
`SecurityTests/`	Security control verification
`Support/`	Test harness: configuration, base class, database access, page objects
`TestCases/`	The 36 test case specifications (JSON and Markdown)
`TRACEABILITY.md`	Case-to-method mapping and defect register
`TestResults/`	Recorded run artefacts